NeuroCheck: analizador de configuraciones de redes neuronales con NeuroLang

Proyecto final — Compiladores, 2026-2
Autor e integrante único: Adrian Alexander Benavides Caguasango
Repositorio: https://github.com/AlexBKG/NeuroCheck
Versión evaluada: v1.2.0 · **Fecha:** 11 de noviembre de 2026

1. Problema y aplicación de compiladores

En una red neuronal con capas densas, las matrices de pesos deben tener dimensiones compatibles con las entradas y las neuronas de cada capa. Para detectar configuraciones incoherentes sin entrenar el modelo, desarrollé **NeuroCheck**, un analizador estático de **NeuroLang**, el lenguaje específico de dominio (DSL) que definí para describir arquitecturas sencillas de redes neuronales.

Por ejemplo:

```
red MiRed {
    entrada 3;
    capa oculta: densa(4, pesos=[3,4]);
    capa salida: densa(2, pesos=[4,2]);
}
```

Con NeuroCheck compruebo la sintaxis de estas declaraciones y la convención `pesos=[entradas,neuronas]`. Mi aplicación no entrena redes, no calcula pesos numéricos ni genera código ejecutable; aplica técnicas de compiladores a la validación de configuraciones.

2. Arquitectura y decisiones de diseño

Organicé el procesamiento en Python y separé la interfaz de la lógica que determina si una configuración es válida.

Componente	Función
`neurocheck/grammar.lark`	Define la gramática de NeuroLang.
`neurocheck/compiler.py`	Reconoce tokens, analiza la sintaxis con Lark/LALR, construye el AST y comprueba reglas semánticas.
`neurocheck/visualization.py`	Representa mediante SVG las capas de una configuración válida.
`neurocheck/cli.py`	Permite analizar archivos `.neuro` desde la terminal.
`app.py` y `web/`	Proveen un servidor HTTP local y una interfaz HTML, CSS y JavaScript para mostrar resultados.

Análisis léxico. El lexer de Lark reconoce palabras reservadas, identificadores, números enteros y signos; produce tokens con tipo, valor, línea y columna. **Análisis sintáctico.** Un parser LALR verifica las producciones de NeuroLang y genera el árbol de análisis. **AST.** Un `Transformer` de Lark convierte ese árbol en una estructura más breve que contiene nombre de red, entradas, capas y dimensiones. **Análisis semántico.** Se revisan declaraciones obligatorias, nombres duplicados, cantidades permitidas y dimensiones de pesos. Se produce una tabla de símbolos de las capas y, si hay errores, diagnósticos con sugerencias.

Elegí Lark para concentrarme en la gramática y las reglas semánticas. Separé el compilador, el servidor y la interfaz para poder ejecutar pruebas sin navegador.

3. Aporte original

Mi aporte principal es el diseño de NeuroLang y de un **verificador de compatibilidad dimensional entre capas densas**. Para una capa con `n` entradas y `m` neuronas, la matriz declarada debe tener dimensiones `[n,m]`. El valor `n` de cada capa posterior corresponde a la cantidad de neuronas de la capa anterior.

Por ejemplo, después de una capa con cuatro neuronas, al escribir `capa salida: densa(2, pesos=[5,2]);` obtengo el diagnóstico **E201**, ya que se esperan pesos `[4,2]`. Complementé esta validación con diagnósticos léxicos y sintácticos y con vistas de tokens, árbol de análisis, AST, tabla de símbolos y arquitectura de la red.

Realicé esta implementación como un proyecto individual de análisis estático, no como una extensión de un compilador de terceros. Utilicé Lark como biblioteca de parsing y desarrollé las validaciones semánticas de NeuroLang para comprobar las dimensiones de las capas.

4. Pruebas y resultados

Ejecuté `python -m pytest -q` sobre la versión 1.2.0 y obtuve **30 pruebas aprobadas**: 18 del compilador, seis del visualizador y seis del servidor HTTP. Los casos cubren ejemplos válidos, tokens, árbol y AST, nombres repetidos, entrada repetida, errores de sintaxis y caracteres, límites de tamaño, dimensiones incompatibles y endpoints del servidor.

Caso	Entrada / condición	Resultado observado
Red válida	Entrada 3; capas 4 y 2; pesos `[3,4]` y `[4,2]`	Configuración aceptada; modelo y diagrama disponibles.
Error semántico	Segunda capa con `pesos=[5,2]`	E201; se esperan dimensiones `[4,2]`.
Error sintáctico	Declaración que no cumple la gramática	E002; no se construye un modelo válido.
Error léxico	Carácter no permitido por NeuroLang	E001; se informa su ubicación.

Para reproducir mis resultados se puede ejecutar `python -m pytest -q` y utilizar los archivos de `examples/`. Estas pruebas no garantizan que todos los programas posibles estén libres de errores.

5. Limitaciones y trabajo futuro

Por ahora, en NeuroLang admito declaraciones sencillas de entrada y capas densas, pero no modelos de PyTorch o TensorFlow. Verifico **dimensiones declaradas**, no valores concretos de matrices, derivadas, inferencia ni entrenamiento. Para conservar la legibilidad, limito la cantidad de neuronas dibujadas. Como trabajo futuro, planteo ampliar las capas admitidas, validar matrices numéricas y exportar arquitecturas a otros formatos.

6. Contribución individual

Desarrollé **NeuroCheck como trabajo individual**. Me encargué del planteamiento, el diseño, el desarrollo, la integración, la verificación, la documentación y la publicación del proyecto:

- **Diseño:** definí el problema, el alcance de NeuroCheck y la sintaxis y las reglas de NeuroLang.
- **Desarrollo e integración:** implementé y revisé los módulos léxico, sintáctico y semántico e integré el servidor local, la interfaz y la visualización de arquitecturas.
- **Verificación y corrección:** ejecuté las pruebas, comprobé casos válidos e inválidos y ajusté los diagnósticos y la presentación.

- **Documentación y publicación:** preparé el README y el reporte técnico, organicé el repositorio y publiqué las versiones.

Utilicé **ChatGPT como herramienta de apoyo** para desarrollar, revisar y corregir partes del código y la documentación. Los cambios del proyecto pueden consultarse en el historial de commits y en la versión v1.2.0.

7. Referencias y herramientas utilizadas

1. Curso de Compiladores. *Proyecto Final Compiladores 2026-2*. Enunciado y rúbrica suministrados por el docente.
2. Lark. *Lark — Parsing Toolkit for Python*. https://lark-parser.readthedocs.io/ (consulta: noviembre de 2026).
3. Python Software Foundation. *Python 3 Documentation*. https://docs.python.org/3/ (consulta: noviembre de 2026).
4. MDN Web Docs. *Web technology for developers* (HTML, CSS, JavaScript). https://developer.mozilla.org/ (consulta: noviembre de 2026).
5. OpenAI. *ChatGPT*. https://chatgpt.com/ (herramienta de apoyo durante el desarrollo y la revisión del código y la documentación).
6. Alina Grubnyak. Fotografía de fondo proporcionada para la interfaz, archivo `alina-grubnyak-ZiQkhI7417A-unsplash.jpg` (Unsplash).